

MyMavenWebApp

MyMavenWebApp with Jenkins

MyMavenWebApp is a simple web application project developed using Apache Maven and integrated with Jenkins in a DevOps environment.

The project demonstrates how Jenkins automates the build and deployment process of a Maven-based web application. Maven is used for project management, dependency handling, compiling source code, testing, and packaging the application, while Jenkins performs Continuous Integration (CI) by automatically building the project whenever changes are made.

Working Process

1. Developer writes and updates the web application code.
2. Source code is stored in a version control system like Git.
3. Jenkins monitors the repository for changes.
4. When new code is pushed, Jenkins automatically triggers the Maven build process.
5. Maven compiles the project, runs tests, and creates a WAR file.
6. Jenkins displays the build status and can deploy the application to a web server.

Features

- Automated build process
- Dependency management using Maven
- Continuous Integration using Jenkins
- Automated testing and deployment
- Easy project management

Advantages

- Reduces manual work
- Faster software delivery
- Early error detection
- Improves software quality
- Supports DevOps automation practices

Applications

- Web application deployment
- Continuous Integration and Continuous Deployment (CI/CD)
- Automated software testing
- DevOps laboratory experiments and project demonstrations

This project helps students understand how Maven and Jenkins work together to automate software development and deployment processes in DevOps environments.

Introduction to Jenkins

Jenkins is an open-source automation server widely used in the field of Continuous Integration (CI) and Continuous Delivery (CD). It allows developers to automate the building, testing, and [deployment](#) of [software](#) projects, making the development process more efficient and reliable.

Key features of Jenkins:

- CI/CD: Jenkins supports Continuous Integration and Continuous [Deployment](#), allowing developers to integrate code changes frequently and automate the deployment of applications.
- Plugins: Jenkins has a vast library of plugins that can extend its capabilities. These plugins integrate Jenkins with version control systems (like Git), build tools (like Maven or Gradle), testing frameworks, deployment tools, and much more.
- Pipeline as Code: Jenkins allows the creation of pipelines using Groovy-based DSL scripts or YAML files, enabling version-controlled and repeatable pipelines.
- Cross-platform: Jenkins can run on various platforms such as Windows, Linux, macOS, and others.

Installing Jenkins Locally

Jenkins can be installed on local machines, on a cloud environment, or even in containers. Here's how you can install Jenkins in Window local System environments:

Step-by-Step Guide

1. Prerequisites:

- Ensure that Java (JDK) is installed on your system. Jenkins requires Java 21. If not then [click here](#).
- You can check if Java is installed by running `java -version` in the terminal.

2. Install Jenkins on Window System):

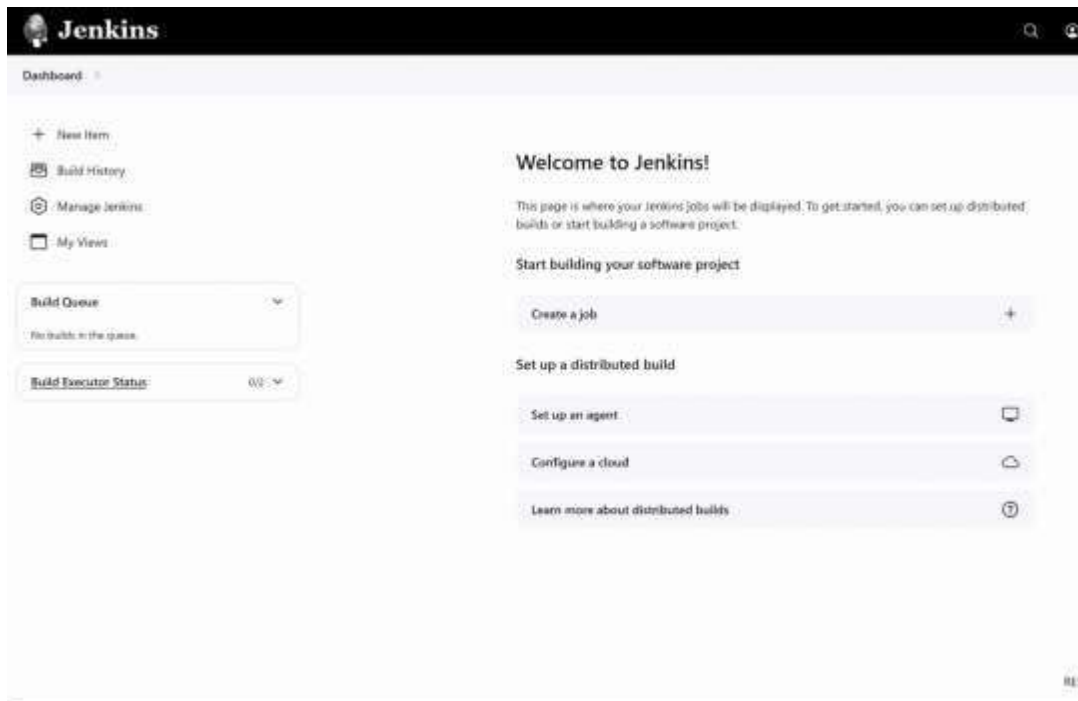
- Download the Jenkins Windows installer from the [official Jenkins website](#).
- Run the installer and follow the on-screen instructions. While installing choose login system: run service as LocalSystem (not recommended).
- After then use default port or you can configure you own port like I'm using port 3030 then click on test and next.
- After then change the directory and choose java jdk-21 path look like C:\Program Files\Java\jdk-21\.
- After then click next, next and then it will ask permission click on yes and it will start installing.
- After successfully installed, Jenkins will be running on port either default port or chosen port like i choose port 3030 by default (you can access it in your browser at `http://localhost:8080`) or `http://localhost:3030`.

Jenkins Setup in browser:

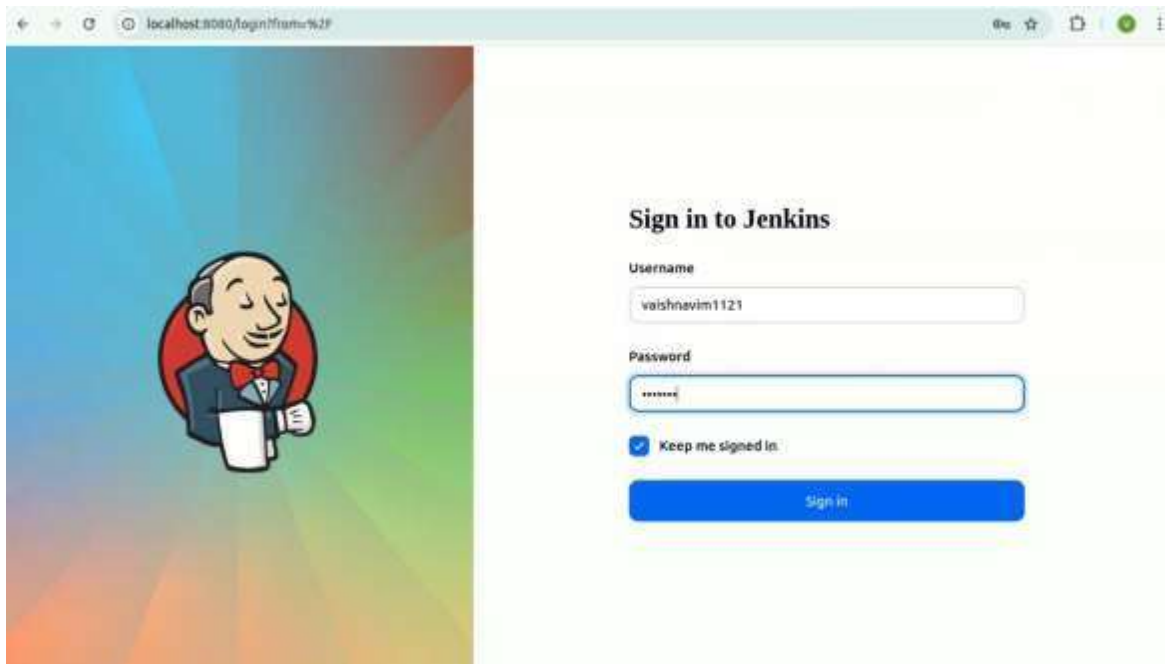
- After opening browser by visiting your local address the browser should look like below screenshot.



- It will ask administrator password so you have to navigate the above highlighted path and open that initialAdminPassword in notepad or any software to see the password.
- Just copy that password and paste it and click on continue.
- It will ask to customize Jenkins so click on install suggested plugin it will automatically install all required plugin.
- After then create admin profile by filling all details then click on save and continue after then save and finish after then click on start using Jenkin



Enter the login credentials to access the dashboard



The image shows the Jenkins login page in a web browser. The address bar displays 'localhost:8080/login?from=%2F'. On the left, there is a large blue and red graphic of the Jenkins mascot. The main content area is titled 'Sign in to Jenkins'. It contains a 'Username' field with the text 'vaishnavim1121', a 'Password' field with masked characters, a checked checkbox for 'Keep me signed in', and a blue 'Sign in' button.

localhost:8080/login?from=%2F

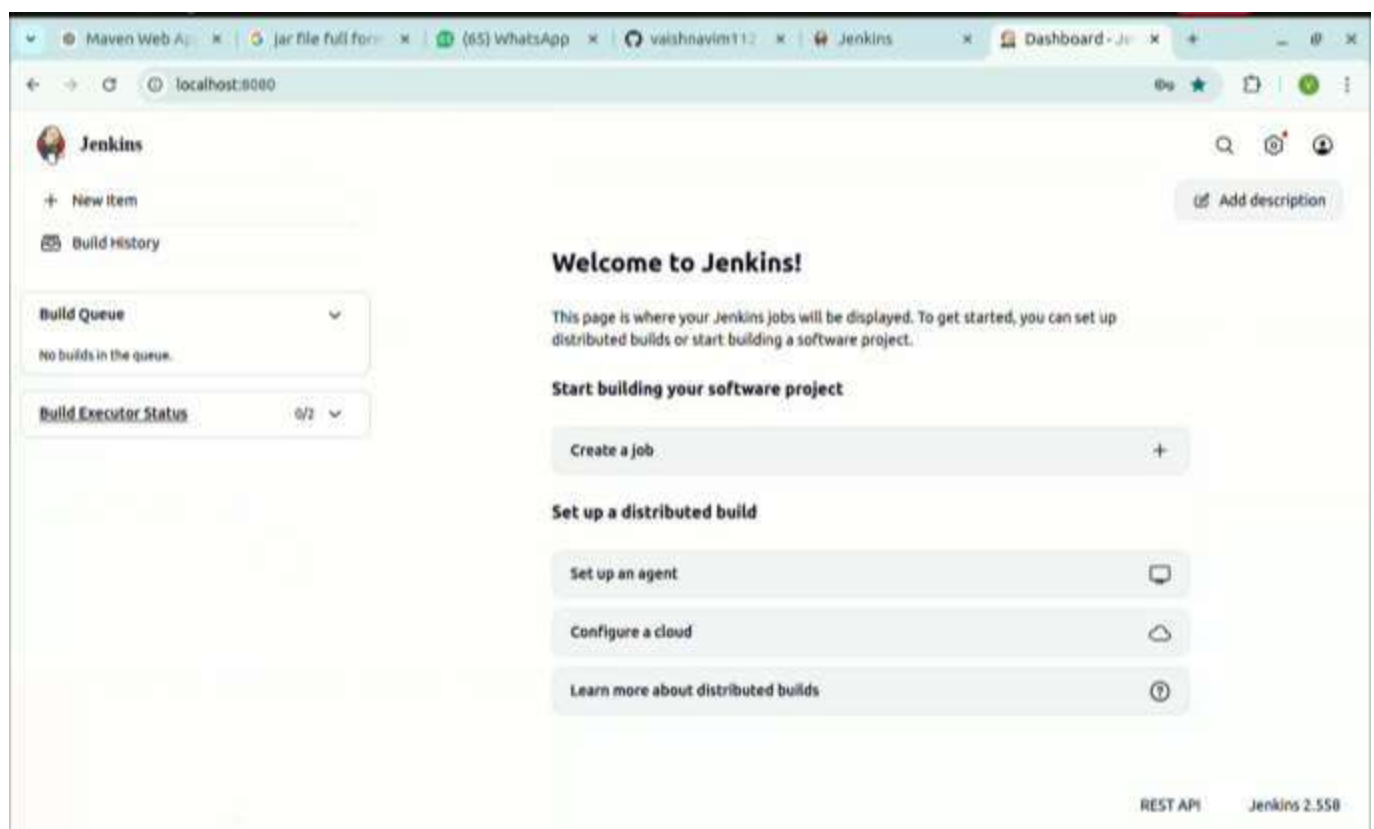
Sign in to Jenkins

Username
vaishnavim1121

Password

☒ Keep me signed in

Sign in



The image shows the Jenkins dashboard in a web browser. The address bar displays 'localhost:8080'. The dashboard has a left sidebar with 'New Item' and 'Build History' links. The main content area is titled 'Welcome to Jenkins!' and includes a description: 'This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.' Below this, there are three sections: 'Start building your software project' with a 'Create a job' button, 'Set up a distributed build' with 'Set up an agent', 'Configure a cloud', and 'Learn more about distributed builds' buttons. The bottom right corner shows 'REST API' and 'Jenkins 2.358'.

localhost:8080

Welcome to Jenkins!

This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.

Start building your software project

Create a job

Set up a distributed build

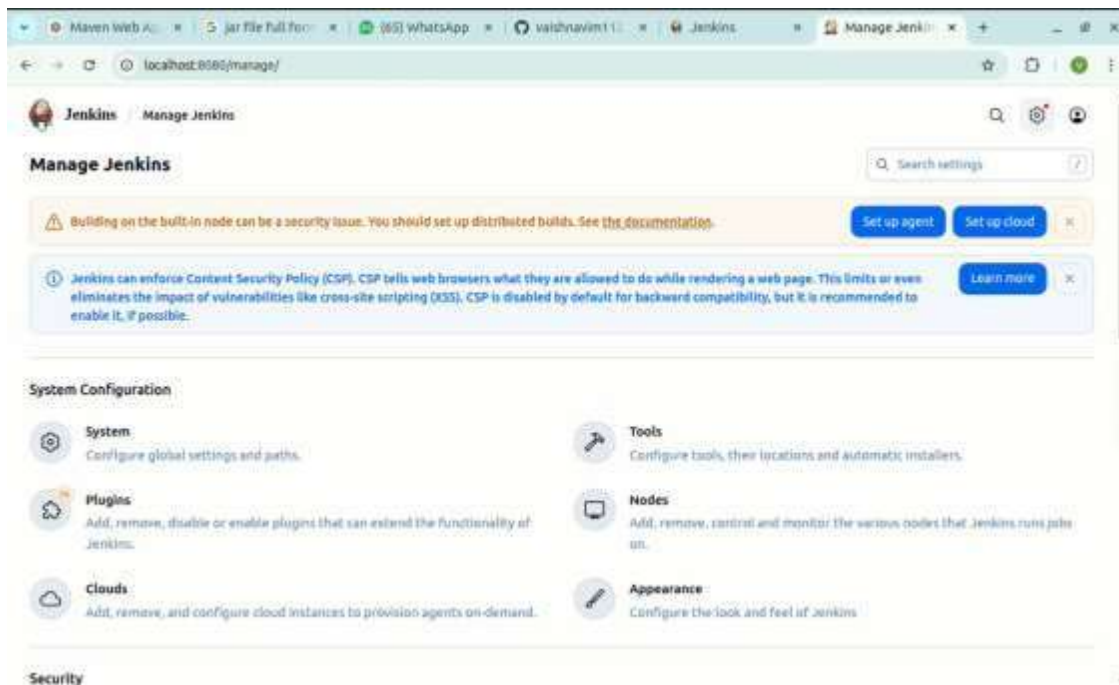
Set up an agent

Configure a cloud

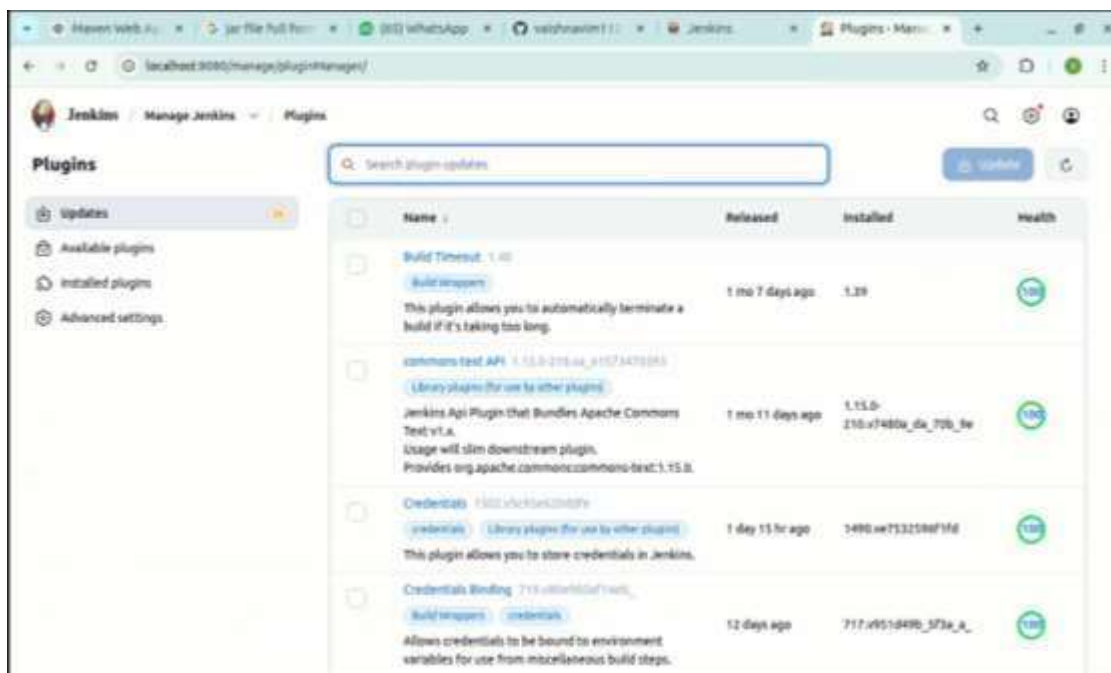
Learn more about distributed builds

REST API Jenkins 2.358

Navigate to Manage Jenkins:



Navigate to Plugins:

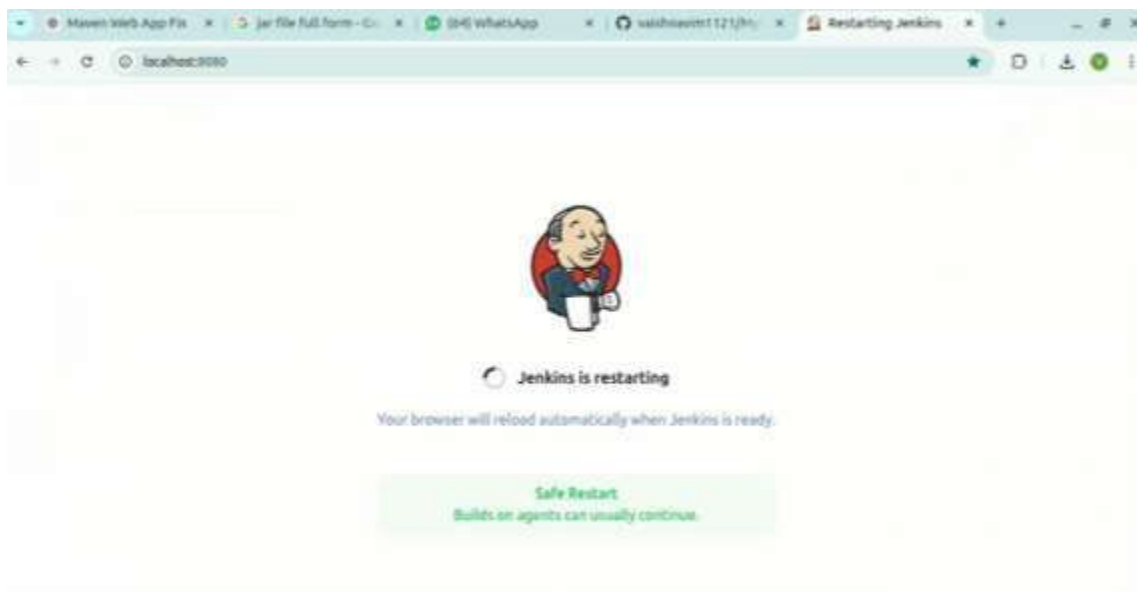
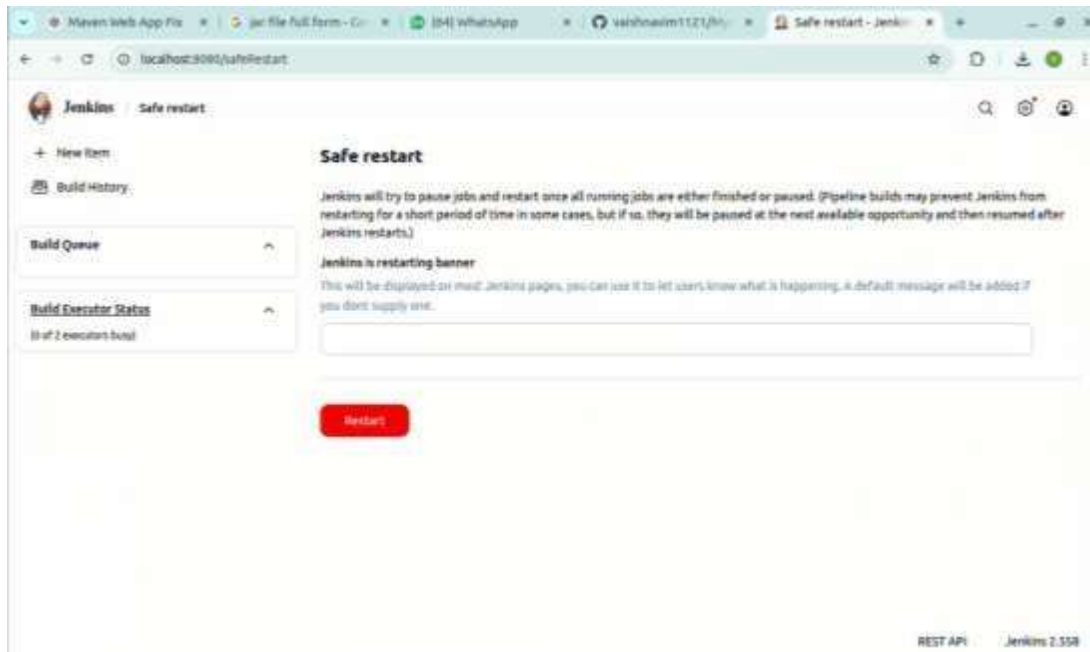


Download the necessary plugins:

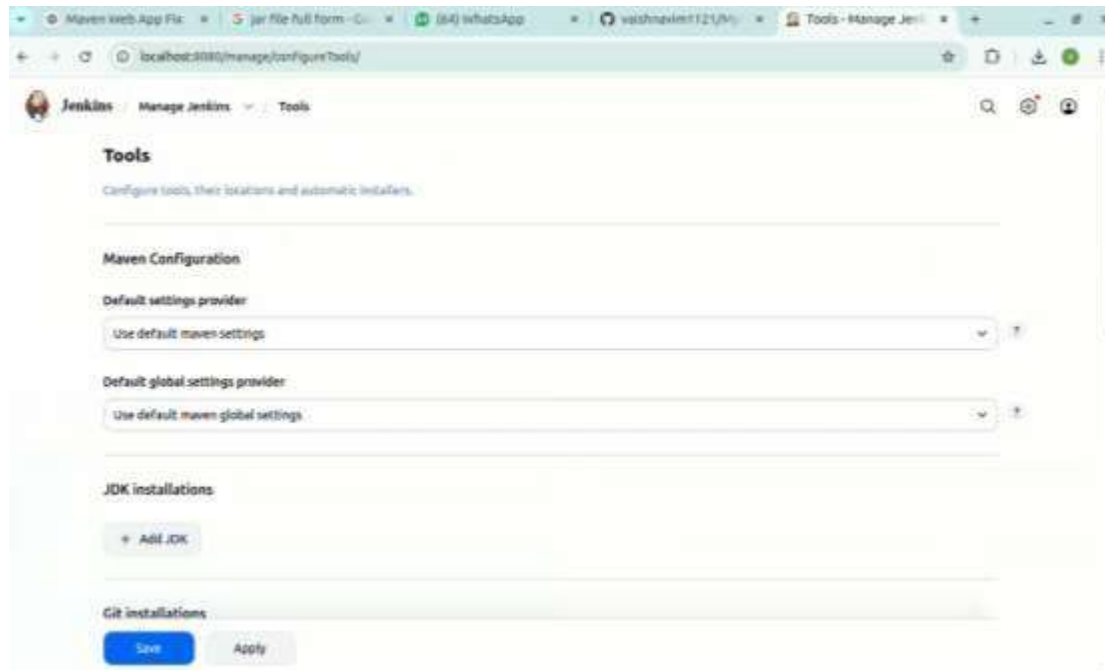
- Maven Integration
- Gradle
- Github Integration
- Pipeline Stage-view Integration

Install the mentioned plugins n verify these downloads in Installed Plugins.

Further, restart the Jenkins using the safeRestart method, i.e, enter the given [url: localhost:8080/safeRestart](http://localhost:8080/safeRestart)



Further, navigate to tools in manage settings



Setup configuration for the downloaded plugins, Add

JDK

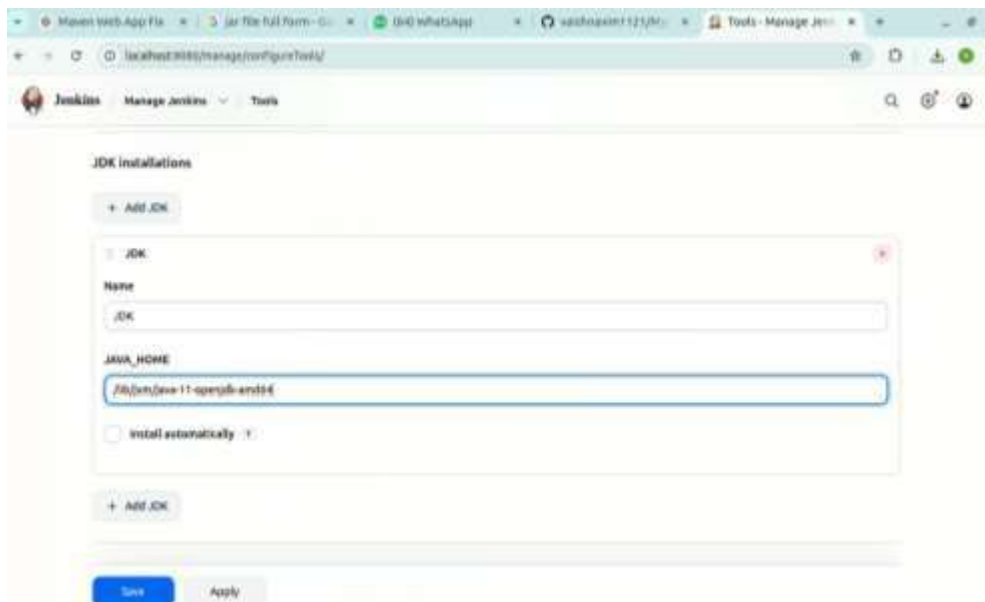
```
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenWebApp1$ which java
/usr/bin/java
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenWebApp1$
```





```
vaishnavi@vaishnavi-virtualbox: /lib/jvm/java-11-openjdk-amd64$
```

Copy the path and paste into Jenkins Tools



Git configuration

```
/usr/bin/java  
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenWebApp1$ which git  
/usr/bin/git  
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenWebApp1$
```


Git installations

Git

Name

Git

Path to Git executable ?

/usr/bin/git

☐ Install automatically ?

+ Add Git

Gradle Configuration

Gradle

name ?

Gradle

GRADLE_HOME ?

/usr/bin/gradle

☐ Install automatically ?

+ Add Gradle

Maven Configuraton

```
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenWebApp1$ which mvn
/usr/bin/mvn
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenWebApp1$
```

Maven

Name

Maven

MAVEN_HOME

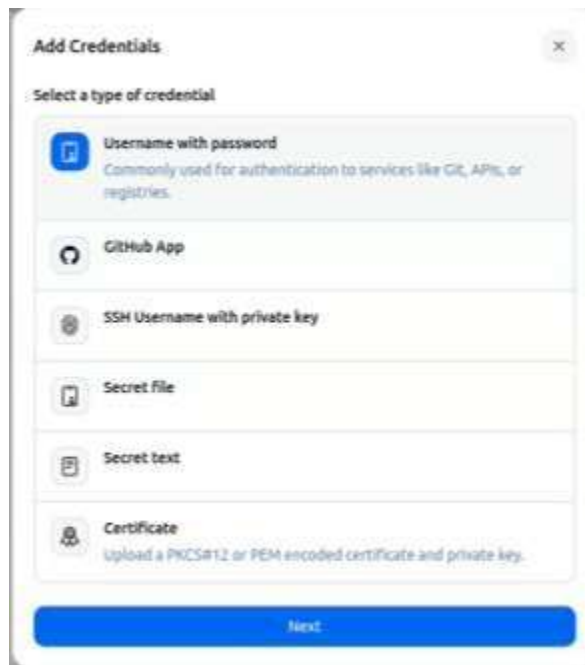
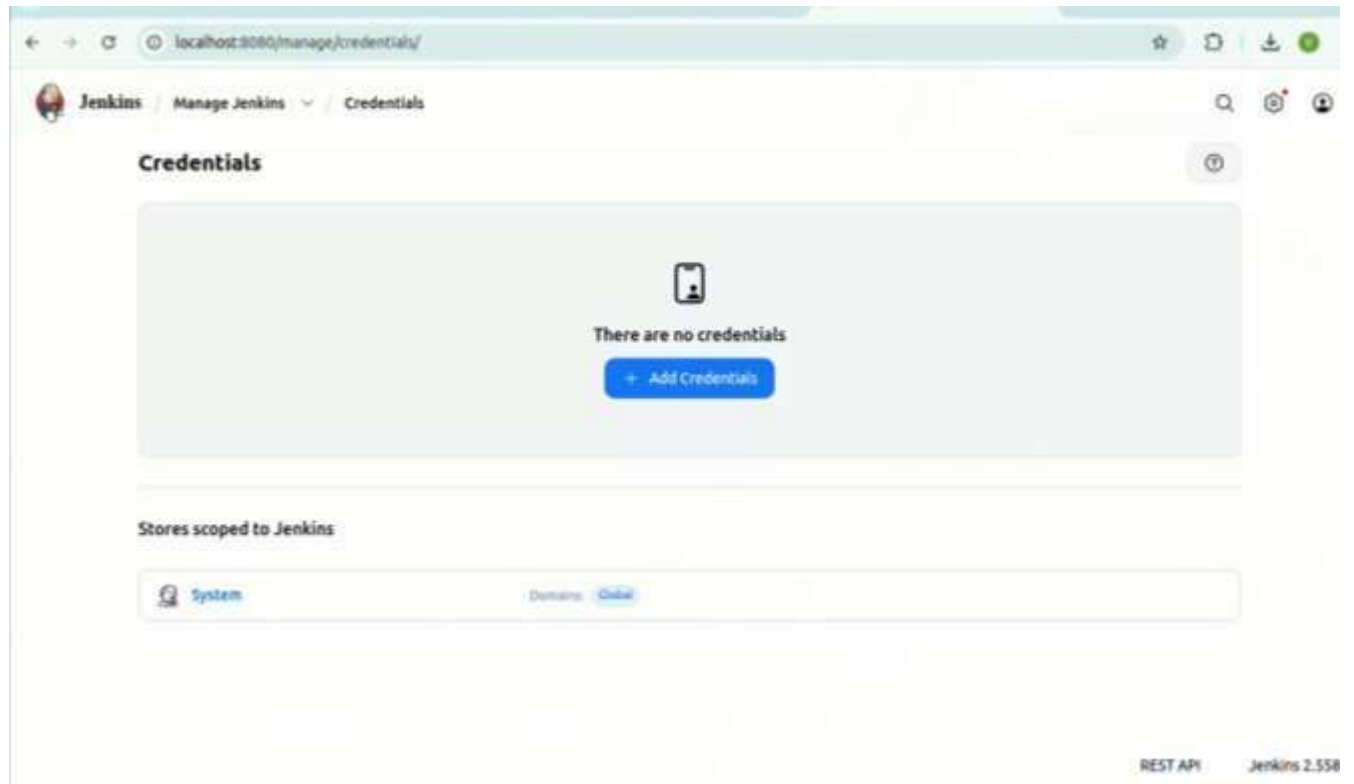
/usr/bin/mvn

☐ Install automatically ?

+ Add Maven

Add Credentials

Manage Jenkins ▾ Credentials



Add username and password of the github credentials

<

Add Username with password

X

Scope ?
Global (Jenkins, nodes, items, all child items, etc) ▼

Username
vaishnavim1121

☒ Treat username as secret ?

Password
XXXXXXXXXXXX

ID ?

Description ?
github ccredentials

Create

localhost:8080/manage/credentials/store/system/domain/_/credential/f797e3e3-933b-4f12-b5de-478252566199/

 **Jenkins**

Manage Jenkins ▼

Credentials / System / Global / github ccredentials

Q ⓘ

Update

Delete

Move

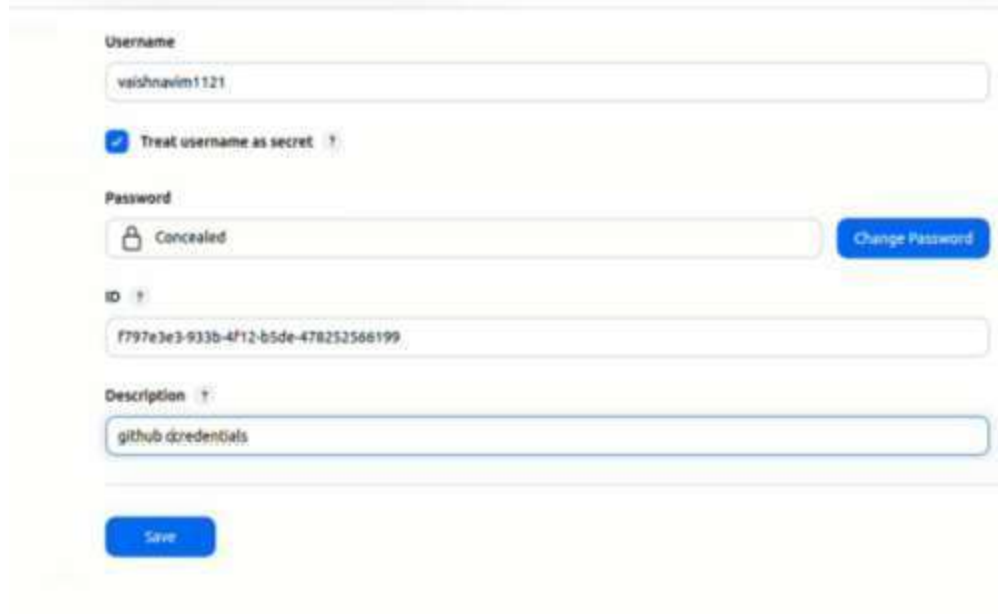
github ccredentials

github ccredentials

Usage

This credential has not been recorded as used anywhere.
Note: usage tracking requires the cooperation of plugins and consequently may not track every use.

REST API Jenkins 2.558



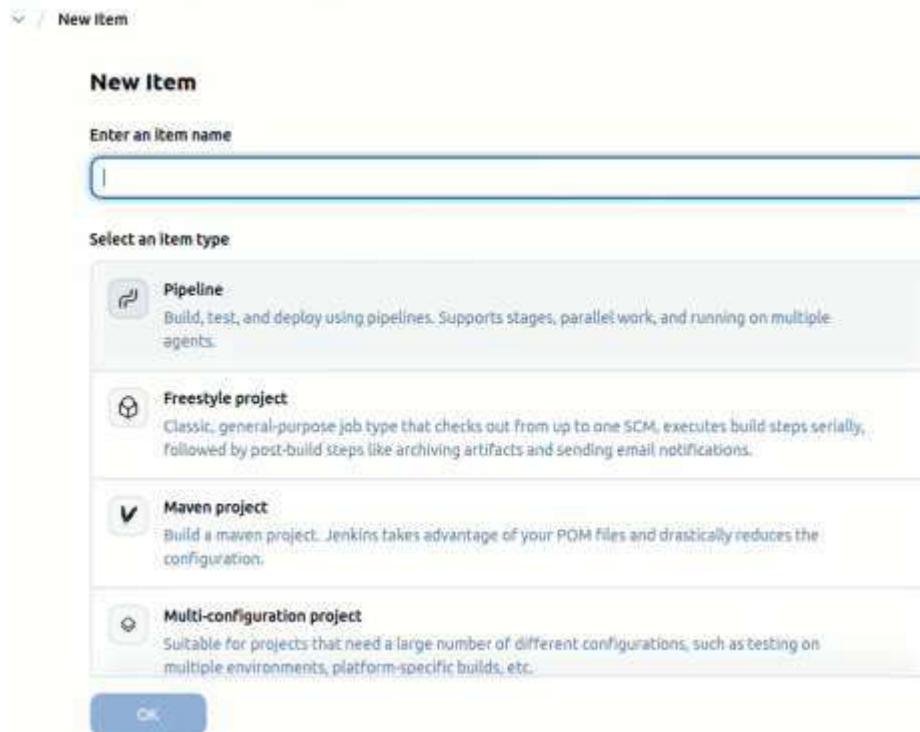
The screenshot shows the Jenkins 'Add Credentials' configuration page. It includes a 'Username' field with the value 'vaishnavin1121', a checked checkbox for 'Treat username as secret', a 'Password' field with a lock icon and the text 'Concealed', a 'Change Password' button, an 'ID' field with the value 'f797e3e3-933b-4f12-b5de-478252568199', and a 'Description' field with the value 'github credentials'. A 'Save' button is at the bottom.

Second Phase

Depicting Jenkins Pipeline

Follow the given steps to deploy the project on Jenkins.

create a Jenkins pipeline job that will use the Jenkinsfile from your project repository
go to the Jenkins home page and click on “New Item” to create a job, enter the item name select Pipeline as the project type.



The screenshot shows the Jenkins 'New Item' configuration page. It has a breadcrumb 'New Item' and a section 'Enter an item name' with an empty text field. Below is a section 'Select an item type' with four options: 'Pipeline' (Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.), 'Freestyle project' (Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.), 'Maven project' (Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.), and 'Multi-configuration project' (Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.). An 'OK' button is at the bottom.

This option allows you to build, test, and deploy applications using a Jenkinsfile. After selecting the pipeline type, click “OK” to proceed to the job configuration page

Definition

Pipeline script from SCM

SCM

Git

Repositories

Repository URL

https://github.com/vaishnavim1121/MyMavenWebApp

Please enter Git repository.

Credentials

github credentials

Advanced

Save Apply

Now we need to add a Jenkinsfile into our project which will consist of the operations that will occur when building the project in Jenkins

It includes step wise instructions guiding at each stage with right commands

```
pipeline {
    agent any // Use any available agent

    tools {
        maven 'Maven' // Ensure this matches the name configured in Jenkins
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'master', url: 'https://github.com/vaishnavim1121/MyMavenWebApp.git'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn clean package' // Run Maven build
            }
        }

        stage('Test') {
            steps {
                sh 'mvn test' // Run unit tests
            }
        }

        stage('Deploy WAR') {
            steps {
                sh 'cp target/MyMavenWebApp01.war /opt/tomcat/webapps/'
            }
        }
    }
}
```

Now try building the project with Jenkins

The screenshot shows the Jenkins web interface for a pipeline named "MyMavenWebApp". The interface includes a left sidebar with navigation options like "Status", "Changes", "Build Now", "Delete Pipeline", "Configure", "Rename", "Full Stage View", "Stages", and "Pipeline Syntax". The main area displays the "Stage View" with a table of stage times and a "Builds" list on the left. A notification at the top says "Screenshot captured".

Stage View

Stage	Declarative: Checkout SCM	Declarative: Tool Install	Checkout	Build	Test	Deploy WAR	Declarative: Post Actions
Average stage times: (full run time: ~1min 5s)	3s	573ms	4s	34s	6s	1s	330ms
Build #1 (Apr 15 09:03)	3s	573ms	4s	34s	6s	1s	330ms

Builds

- Filter
- 15 April 2028
- #1 09:03

Permalinks

- Last build (#1), 24 days ago
- Last stable build (#1), 24 days ago
- Last successful build (#1), 24 days ago
- Last completed build (#1), 24 days ago

REST API Jenkins 2.562

The project is successfully built.